

Cours Flutter — Application POS Débits de Boissons

Bases du langage Dart et du framework Flutter, appliquées au projet
Front-boisson

Support de cours — Équipe Front-boisson

Juillet 2026

Contents

1. Introduction	3
1.1 Qu'est-ce que Flutter ?	3
1.2 Qu'est-ce que Dart ?	3
1.3 Le principe fondamental de Flutter : tout est un Widget	3
2. Les bases du langage Dart	4
2.1 Variables et types	4
2.2 Fonctions	4
2.3 Classes	4
2.4 Programmation asynchrone : Future et async/await	5
3. Anatomie d'une application Flutter	6
3.1 Le point d'entrée : main.dart	6
3.2 StatelessWidget vs StatefulWidget	6
4. Widgets de mise en page ("le CSS de Flutter")	8
4.1 Column et Row	8
4.2 Container — l'équivalent d'une <div> stylée	8
4.3 Expanded et ListView	8
5. Widgets d'interaction et formulaires	10
5.1 Champs de texte et boutons	10
5.2 Formulaire avec validation (Form + TextFormField)	10
6. Navigation entre écrans	12
7. Gestion d'état avec Provider	13
7.1 Pourquoi une gestion d'état ?	13
7.2 Créer un Provider	13
7.3 Brancher le Provider sur l'application	13
7.4 Consommer le Provider dans un écran	14
8. Appels API vers le backend Django	15

8.1 Le service HTTP de base	15
8.2 Convertir le JSON en objets Dart (fromJson / toJson)	15
8.3 Le service métier complet	16
9. Le thème et les couleurs (charte graphique du cahier des charges)	17
10. Exemple complet : mini tableau de bord	18
11. Erreurs fréquentes de débutant	20
12. Commandes Flutter essentielles (rappel)	21

1. Introduction

1.1 Qu'est-ce que Flutter ?

Flutter est un framework créé par Google pour construire des applications à partir d'une **seule base de code**, capable de tourner sur mobile (Android/iOS), web, et desktop (Windows/Linux/macOS). C'est pour cette raison qu'il a été retenu pour le projet : la même application Flutter pourra tourner sur les tablettes de caisse, les ordinateurs de bureau et les terminaux tactiles mentionnés dans le cahier des charges (§4.4).

1.2 Qu'est-ce que Dart ?

Dart est le langage de programmation utilisé par Flutter. Il est orienté objet, fortement typé, et compilé — ce qui le rapproche davantage de Java ou C# que de JavaScript ou Python dans sa syntaxe. Si tu connais Python ou PHP (comme sur tes projets Flask/Django), les concepts de classes, fonctions et boucles seront familiers ; la syntaxe change, mais la logique reste la même.

1.3 Le principe fondamental de Flutter : tout est un Widget

En Flutter, **absolument tout ce qui s'affiche à l'écran est un Widget** : un texte, un bouton, une image, une marge, une couleur de fond, un écran entier. Un widget est une classe Dart qui décrit "à quoi ressemble" une portion d'interface. Les widgets s'emboîtent les uns dans les autres pour former un arbre — l'**arbre de widgets**.

C'est l'équivalent conceptuel du DOM en HTML/CSS, sauf qu'il n'y a pas de fichier séparé pour la structure (HTML), le style (CSS) et le comportement (JS) : tout est unifié dans un seul langage, Dart.

2. Les bases du langage Dart

2.1 Variables et types

```
String nom = "Bière Castel";           // texte
int quantite = 24;                      // entier
double prix = 500.0;                   // décimal
bool actif = true;                     // booléen
var prixVente = 750.0;                  // type déduit automatiquement (var)
final dateCreation = DateTime.now();    // valeur assignée une seule fois
const tva = 0.1925;                    // constante fixée à la compilation
```

Nullabilité : Dart distingue les valeurs qui peuvent être null de celles qui ne le peuvent pas. C'est une source d'erreurs fréquente au début.

```
String nom = "Boisson";                // ne peut jamais être null
String? description;                   // le "?" autorise null
description = null;                    // valide
nom = null;                            // ERREUR de compilation
```

2.2 Fonctions

```
double calculerSousTotal(int quantite, double prixUnitaire) {
    return quantite * prixUnitaire;
}

// Version compacte (fonction fléchée)
double calculerSousTotal2(int quantite, double prixUnitaire) => quantite * prixUnitaire;
```

2.3 Classes

Les classes Dart ressemblent aux classes Python, avec un constructeur explicite.

```
class Produit {
    final int id;
    final String nom;
    final double prixVente;

    Produit({required this.id, required this.nom, required this.prixVente});

    // Méthode
    String afficherPrix() => "$prixVente FCFA";
}

// Utilisation
final produit = Produit(id: 1, nom: "Coca-Cola", prixVente: 500);
print(produit.afficherPrix());
```

required signifie que ce paramètre est obligatoire à la création de l'objet. C'est le pattern utilisé pour **tous les modèles** du projet (Produit, Vente, Client, etc. dans lib/models/).

2.4 Programmation asynchrone : Future et async/await

C'est le concept Dart le plus important pour ce projet, car **chaque appel à l'API Django est asynchrone** (le résultat n'arrive pas instantanément).

```
Future<String> recupererDonnees() async {
  // Simule un appel réseau qui prend du temps
  await Future.delayed(Duration(seconds: 2));
  return "Données reçues";
}

void main() async {
  print("Début");
  String resultat = await recupererDonnees();
  print(resultat); // s'affiche seulement après 2 secondes
  print("Fin");
}
```

- Future<T> = "une valeur de type T qui sera disponible plus tard" (équivalent d'une Promise JS ou d'une coroutine Python asyncio).
- async = marque une fonction comme asynchrone.
- await = attend le résultat sans bloquer le reste de l'application.

C'est exactement ce schéma qui sera utilisé dans lib/services/produit_service.dart pour appeler GET /api/produits/ sur le backend Django.

3. Anatomie d’une application Flutter

3.1 Le point d’entrée : main.dart

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Front Boisson',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: const LoginScreen(),
    );
  }
}
```

- `runApp()` lance l’application avec le widget racine.
- `MaterialApp` fournit le thème global, la navigation, et le style Material Design (Google) à toute l’app.
- `home`: définit le premier écran affiché — dans notre projet, ce sera `LoginScreen`.

3.2 StatelessWidget vs StatefulWidget

C’est la distinction la plus importante à comprendre en Flutter.

StatelessWidget — l’affichage ne change jamais après sa construction (pas de mémoire interne) :

```
class StatCard extends StatelessWidget {
  final String titre;
  final String valeur;

  const StatCard({super.key, required this.titre, required this.valeur});

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Column(
        children: [
          Text(titre),
          Text(valeur, style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold)),
        ],
      ),
    );
  }
}
```

```

    ),
  );
}
}

```

StatefulWidget — l’affichage peut changer en réaction à une action (saisie utilisateur, réponse API, minuteur...) :

```

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  bool _motDePasseVisible = false; // variable "mémoire" de l'écran

  void _toggleVisibilite() {
    setState(() { // déclenche un redessin de l'écran
      _motDePasseVisible = !_motDePasseVisible;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: IconButton(
        icon: Icon(_motDePasseVisible ? Icons.visibility : Icons.visibility_off),
        onPressed: _toggleVisibilite,
      ),
    );
  }
}

```

Règle pratique : si l’écran a un formulaire, un bouton qui change d’état visuel, ou des données qui arrivent après un appel API, c’est un StatefulWidget. Sinon, StatelessWidget suffit et est plus performant.

4. Widgets de mise en page (“le CSS de Flutter”)

Flutter n’a pas de CSS séparé : la disposition se fait en emboîtant des widgets de mise en page, directement en Dart.

4.1 Column et Row

```
Column(// empile verticalement
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    Text("Ligne 1"),
    Text("Ligne 2"),
  ],
)

Row(// aligne horizontalement
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  children: [
    Icon(Icons.local_bar),
    Text("Bière"),
  ],
)
```

4.2 Container — l’équivalent d’une <div> stylée

```
Container(
  padding: const EdgeInsets.all(16),
  margin: const EdgeInsets.symmetric(vertical: 8),
  decoration: BoxDecoration(
    color: Colors.white,
    borderRadius: BorderRadius.circular(12),
    boxShadow: [BoxShadow(color: Colors.black12, blurRadius: 4)],
  ),
  child: const Text("Carte produit"),
)
```

4.3 Expanded et ListView

```
// Expanded : fait grandir un widget pour occuper l'espace restant
Row(
  children: [
    Expanded(child: Text("Nom du produit")),
    Text("500 FCFA"),
  ],
)
```



```
// ListView.builder : liste défilante performante (essentiel pour catalogue produits, historique)
ListView.builder(
  itemCount: produits.length,
  itemBuilder: (context, index) {
    final produit = produits[index];
    return ListTile(
      title: Text(produit.nom),
      trailing: Text("${produit.prixVente} FCFA"),
    );
  },
)
```

ListView.builder sera le widget principal de `produits_screen.dart` et du catalogue dans `pos_screen.dart` : il ne construit que les éléments visibles à l'écran, ce qui reste rapide même avec un catalogue de plusieurs centaines de produits.

5. Widgets d'interaction et formulaires

5.1 Champs de texte et boutons

```
final TextEditingController _emailController = TextEditingController();

TextField(
  controller: _emailController,
  decoration: const InputDecoration(
    labelText: "Nom d'utilisateur",
    border: OutlineInputBorder(),
  ),
)

ElevatedButton(
  onPressed: () {
    print(_emailController.text); // récupère la valeur saisie
  },
  child: const Text("Se connecter"),
)
```

5.2 Formulaire avec validation (Form + TextFormField)

```
final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: Column(
    children: [
      TextFormField(
        decoration: const InputDecoration(labelText: "Email"),
        validator: (value) {
          if (value == null || value.isEmpty) return "Champ requis";
          if (!value.contains('@')) return "Email invalide";
          return null; // null = pas d'erreur
        },
      ),
      ElevatedButton(
        onPressed: () {
          if (_formKey.currentState!.validate()) {
            // toutes les validations sont passées
          }
        },
        child: const Text("Valider"),
      ),
    ],
  ),
)
```

```
)
```

Ce pattern (`_formKey`, `validator`) sera utilisé dans `login_screen.dart` et `produit_form_screen.dart`.

6. Navigation entre écrans

Flutter propose deux approches ; le projet utilise les **routes nommées**, déclarées dans `lib/routes/app_routes.dart`.

```
// Déclaration des routes (dans MaterialApp)
MaterialApp(
  initialRoute: '/login',
  routes: {
    '/login': (context) => const LoginScreen(),
    '/dashboard': (context) => const DashboardScreen(),
    '/pos': (context) => const PosScreen(),
  },
)

// Naviguer VERS un écran
Navigator.pushNamed(context, '/dashboard');

// Naviguer et empêcher le retour en arrière (utile après un login réussi)
Navigator.pushReplacementNamed(context, '/dashboard');

// Revenir à l'écran précédent
Navigator.pop(context);
```

7. Gestion d'état avec Provider

7.1 Pourquoi une gestion d'état ?

Sans outil dédié, partager une donnée (ex : "l'utilisateur est-il connecté ?", "que contient le panier ?") entre plusieurs écrans obligerait à la faire transiter manuellement de widget en widget. **Provider** résout ce problème en rendant une donnée accessible depuis n'importe quel écran de l'arbre.

7.2 Créer un Provider

```
// lib/providers/auth_provider.dart
import 'package:flutter/foundation.dart';

class AuthProvider extends ChangeNotifier {
  String? _token;
  String? _nomUtilisateur;

  bool get estConnecte => _token != null;
  String? get nomUtilisateur => _nomUtilisateur;

  Future<void> connecter(String nomUtilisateur, String motDePasse) async {
    // appel à AuthService ici (voir chapitre 8)
    _token = "faux-token-recu-de-django";
    _nomUtilisateur = nomUtilisateur;
    notifyListeners(); // prévient TOUS les écrans qui écoutent ce provider
  }

  void deconnecter() {
    _token = null;
    _nomUtilisateur = null;
    notifyListeners();
  }
}
```

ChangeNotifier + notifyListeners() est le mécanisme central : chaque fois qu'une donnée change, tous les widgets abonnés sont automatiquement redessinés.

7.3 Brancher le Provider sur l'application

```
// main.dart
void main() {
  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => AuthProvider()),
        ChangeNotifierProvider(create: (_) => DashboardProvider()),
      ],
    ),
  );
}
```

```

        ChangeNotifierProvider(create: (_) => PanierProvider()),
    ],
    child: const MyApp(),
  ),
);
}

```

7.4 Consommer le Provider dans un écran

```

// Lecture réactive (l'écran se redessine si la donnée change)
Widget build(BuildContext context) {
  final auth = context.watch<AuthProvider>();
  return Text(auth.estConnecte ? "Bonjour ${auth.nomUtilisateur}" : "Non connecté");
}

// Action, sans avoir besoin de redessiner (dans un onPressed par exemple)
onPressed: () {
  context.read<AuthProvider>().connecter(email, motDePasse);
}

```

Règle simple : `context.watch` dans le `build()` pour afficher une donnée, `context.read` dans un callback (`onPressed`, `onTap`...) pour déclencher une action.

8. Appels API vers le backend Django

8.1 Le service HTTP de base

```
// lib/services/api_service.dart
import 'dart:convert';
import 'package:http/http.dart' as http;

class ApiService {
  static const String baseUrl = "https://sport.services-ztf.com/api"; // exemple, à adapter

  Future<dynamic> get(String endpoint) async {
    final response = await http.get(Uri.parse('$baseUrl$endpoint'));
    if (response.statusCode == 200) {
      return jsonDecode(response.body);
    } else {
      throw Exception('Erreur API : ${response.statusCode}');
    }
  }

  Future<dynamic> post(String endpoint, Map<String, dynamic> data) async {
    final response = await http.post(
      Uri.parse('$baseUrl$endpoint'),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode(data),
    );
    if (response.statusCode == 200 || response.statusCode == 201) {
      return jsonDecode(response.body);
    } else {
      throw Exception('Erreur API : ${response.statusCode}');
    }
  }
}
```

8.2 Convertir le JSON en objets Dart (fromJson / toJson)

C'est le pont entre les tables PostgreSQL (via Django REST Framework) et les classes Dart de lib/models/.

```
// lib/models/produit.dart
class Produit {
  final int id;
  final String nom;
  final double prixVente;
  final int idCategorie;

  Produit({
    required this.id,
```

```

    required this.nom,
    required this.prixVente,
    required this.idCategorie,
  });

// Construit un Produit à partir du JSON renvoyé par l'API Django
factory Produit.fromJson(Map<String, dynamic> json) {
  return Produit(
    id: json['id_produit'],
    nom: json['nom'],
    prixVente: double.parse(json['prix_vente'].toString()),
    idCategorie: json['id_categorie'],
  );
}

// Transforme un Produit en JSON pour l'envoyer à l'API (création/modification)
Map<String, dynamic> toJson() {
  return {
    'nom': nom,
    'prix_vente': prixVente,
    'id_categorie': idCategorie,
  };
}
}

```

8.3 Le service métier complet

```

// lib/services/produit_service.dart
class ProduitService {
  final ApiService _api = ApiService();

  Future<List<Produit>> getProduits() async {
    final data = await _api.get('/produits/');
    return (data as List).map((json) => Produit.fromJson(json)).toList();
  }

  Future<Produit> creerProduit(Produit produit) async {
    final data = await _api.post('/produits/', produit.toJson());
    return Produit.fromJson(data);
  }
}

```


9. Le thème et les couleurs (charte graphique du cahier des charges)

```
// lib/core/theme/app_colors.dart
import 'package:flutter/material.dart';

class AppColors {
  static const bleuFonce = Color(0xFF0D1B4C); // navigation, en-têtes
  static const orange    = Color(0xFFFF7A00); // actions principales
  static const vert      = Color(0xFF2ECC71); // validation
  static const jaune     = Color(0xFFFF1C40F); // alerte
  static const rouge     = Color(0xFFE74C3C); // erreur
  static const fond      = Color(0xFFFF5F6FA); // fond général
}

// lib/core/theme/app_theme.dart
import 'package:flutter/material.dart';
import 'app_colors.dart';

class AppTheme {
  static ThemeData get theme {
    return ThemeData(
      scaffoldBackgroundColor: AppColors.fond,
      primaryColor: AppColors.bleuFonce,
      appBarTheme: const AppBarTheme(
        backgroundColor: AppColors.bleuFonce,
        foregroundColor: Colors.white,
      ),
      elevatedButtonTheme: ElevatedButtonThemeData(
        style: ElevatedButton.styleFrom(
          backgroundColor: AppColors.orange,
          foregroundColor: Colors.white,
          padding: const EdgeInsets.symmetric(vertical: 14, horizontal: 24),
          shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(8)),
        ),
      ),
    );
  }
}
```

Ce thème est ensuite branché une seule fois dans `main.dart` (`MaterialApp(theme: AppTheme.theme)`) et s'applique automatiquement à tous les écrans.

10. Exemple complet : mini tableau de bord

Cet exemple assemble tous les concepts vus (Provider, appel API, widgets, thème) dans une version simplifiée de `dashboard_screen.dart`.

```
// lib/providers/dashboard_provider.dart
class DashboardProvider extends ChangeNotifier {
  double chiffreAffairesJour = 0;
  int nombreVentes = 0;
  bool chargement = false;

  Future<void> chargerDonnees() async {
    chargement = true;
    notifyListeners();

    final data = await DashboardService().getStatsJour();
    chiffreAffairesJour = data['chiffre_affaires'];
    nombreVentes = data['nombre_ventes'];

    chargement = false;
    notifyListeners();
  }
}

// lib/screens/dashboard/dashboard_screen.dart
class DashboardScreen extends StatefulWidget {
  const DashboardScreen({super.key});

  @override
  State<DashboardScreen> createState() => _DashboardScreenState();
}

class _DashboardScreenState extends State<DashboardScreen> {
  @override
  void initState() {
    super.initState();
    // Charge les données dès l'ouverture de l'écran
    Future.microtask(() => context.read<DashboardProvider>().chargerDonnees());
  }

  @override
  Widget build(BuildContext context) {
    final dashboard = context.watch<DashboardProvider>();

    return Scaffold(
      appBar: AppBar(title: const Text("Tableau de bord")),
      body: dashboard.chargement
        ? const Center(child: CircularProgressIndicator())
        : Padding(
```

```

padding: const EdgeInsets.all(16),
child: Row(
  children: [
    Expanded(
      child: StatCard(
        titre: "CA du jour",
        valeur: "${dashboard.chiffreAffairesJour} FCFA",
      ),
    ),
    const SizedBox(width: 16),
    Expanded(
      child: StatCard(
        titre: "Ventes",
        valeur: "${dashboard.nombreVentes}",
      ),
    ),
  ],
),
);
}
}

```

11. Erreurs fréquentes de débutant

Erreur	Cause	Solution
<code>setState()</code> called after <code>dispose()</code>	L'écran a été fermé pendant un appel réseau en cours	Vérifier <code>if (mounted)</code> avant <code>setState()</code>
Null check operator used on a null value	Utilisation de <code>!</code> sur une variable qui vaut null	Vérifier avec <code>if (variable != null)</code> avant
<code>RenderFlex</code> overflowed	Un widget dépasse l'espace disponible	Entourer d'un <code>Expanded</code> , <code>Flexible</code> , ou <code>SingleChildScrollView</code>
L'écran ne se met pas à jour	Modification d'une variable sans <code>setState()</code> ou <code>notifyListeners()</code>	Toujours encapsuler les changements d'état
<code>type 'Null' is not a subtype of type 'String'</code>	Le JSON de l'API ne contient pas le champ attendu	Vérifier la réponse de l'API Django, gérer les champs optionnels avec <code>?</code>
L'app ne recompile pas après un changement	Hot reload insuffisant (changement de structure de classe)	Faire un Hot Restart (touche <code>R</code> majuscule dans le terminal)

12. Commandes Flutter essentielles (rappel)

```
flutter doctor           # diagnostic de l'installation
flutter pub get          # installe les dépendances de pubspec.yaml
flutter run -d chrome    # lance l'app dans Chrome (le plus rapide pour développer)
flutter run              # lance sur l'appareil/émulateur connecté
r                        # (dans le terminal flutter run) hot reload
R                        # (dans le terminal flutter run) hot restart
q                        # quitte le mode run
flutter clean            # nettoie le cache de build (en cas de bug étrange)
flutter pub outdated     # liste les dépendances qui ont une version plus récente
flutter analyze          # vérifie le code (erreurs, warnings de style)
```

Ce cours couvre les bases nécessaires pour contribuer aux modules Connexion et Tableau de bord du projet Front-boisson. Le guide de programmation (document séparé) détaille l'installation, le workflow Git et le plan de développement du jour.